Lesson 6

Material

https://github.com/unixfile/learning-python-2026

Today

- Exception handling
- Reading stack traces
- Regular expressions with the `re` module

1 Exception handling

```
numbers = [1, 2, 3]

try:
    print(numbers[5])
except IndexError:
    print("Index out of range")
```

- `try` runs the block; if an exception is raised Python jumps to `except`
- Code after the raising line inside `try` is skipped

2 Multiple exception types

```
try:
    value = int("abc")
    numbers = [1, 2, 3]
    print(numbers[5])
except ValueError:
    print("Not a valid number")
except IndexError:
    print("Index out of range")

# Catch several types in one handler
try:
    ...
except (ValueError, IndexError) as e:
    print(f"An error occurred: {e}")
```

3 Stack traces

```
Traceback (most recent call last):
  File "example.py", line 10, in <module>
    main()
  File "example.py", line 7, in main
    process_data()
  File "example.py", line 4, in process_data
    value = int("abc")
ValueError: invalid literal for int() with base 10: 'abc'
```

- The last line tells you what went wrong
- The trace shows the call chain from outermost to innermost
- Read from the bottom up, where the error usually is

4 Regular expressions

- A character sequence that defines a search pattern
- Used to search, or search-and-replace, in text
- Widely used and not unique to Python
- Powerful, but quickly get hard to read

Recommended tool

https://regex101.com

5 A first match

```
import re

anthem = "God save our gracious King"

m = re.search(r'gracious King', anthem)
m.group()   # 'gracious King'
```

- `re.search` returns a `Match` for the **first** match, or `None`
- `.group()` returns the matched text
- `r'...'` is a raw string: backslashes stay literal, so regex escapes like `\d` reach the engine unchanged

6 Common patterns

```
.          any character except newline
[abc]      any one of a, b, c
[^abc]     any character except a, b, c
\d         any digit (0-9)
\w         any word character (letter, digit, _)
\s         any whitespace (space, \t, \n)
?          0 or 1 repetition
+          1 or more repetitions
*          0 or more repetitions
^  $       start / end of string
( )        group patterns
a|b        either a or b
\.         a literal dot (escape special chars)
```

7 Greedy vs. non-greedy

```
line = "God save our gracious King"

# Greedy: longest possible match
re.search(r'G.*o', line).group()
# 'God save our gracio'

# Non-greedy: shortest possible match
re.search(r'G.*?o', line).group()
# 'Go'
```

- ?, +, * are greedy; add ? after them to go non-greedy
- Greedy matching can be expensive on large strings

8 Finding all matches

```
text = "The rain in Spain stays mainly in the plain."

# Whole words ending in 'ain'
re.findall(r'\b\w*ain\b', text)
# Output: ['rain', 'Spain', 'plain']
```

- `re.findall` returns every non-overlapping match as a list
- `\b` is a word boundary, so `mainly` is skipped (no boundary after `ain`)

9 Groups

```
text = 'Berlin 1989-11-09 22:45 UTC+1'
rgx  = r'(\d{4})-(\d{2})-(\d{2})\s(\d{2}):(\d{2})'

m = re.search(rgx, text)
m.group(0)   # '1989-11-09 22:45'
m.group(1)   # '1989'
m.group(2)   # '11'
m.group(3)   # '09'
```

- `( )` captures part of a match into a numbered group
- `group(0)` is the whole match, same as `group()`
- Named groups, flags and `re.escape`: see Further reading

Further reading: named groups

```
text = "Date: 2024-06-13"
rgx  = r'(?P<year>\d{4})-(?P<month>\d{2})'

m = re.search(rgx, text)
if m:
    m.group('year')     # '2024'
    m.group('month')    # '06'
```

- `(?P<name>...)` lets you reference a group by name instead of number

Further reading: flags and escaping

```
text = "April is the\ncruellest month"

# By default '.' does not match newline
re.search(r'.*', text).group()
# 'April is the'

# The DOTALL flag changes that
re.search(r'.*', text, flags=re.DOTALL).group()
# 'April is the\ncruellest month'

re.escape("$^*+?{}")
# '\\$\\^\\*\\+\\?\\{\\}'
```

- `re.escape` escapes every special character in a string